Children in a Circle

Lab 3: hill climbing and an evolutionary algorithm

Computational Intelligence 2026-27

The problem

- N children, each with C boolean characteristics
- Difference of two children: number of characteristics that differ
- The children stand in a circle, holding hands
- Goal: **maximize the sum of the differences between adjacent children**

Example, C = 6

```
child A:  +  +  -  +  -  -
child B:  +  -  -  +  +  -
             ^        ^

difference = 2
```

Solution: a permutation

- The list of the children in circle order
- Each child appears exactly once: no illegal solutions
- The same circle can be written in 2n ways (n rotations × 2 directions)
- Only one is kept: the **canonical form**

```
def normalize(solution):
    sol = list(solution)
    i = sol.index(min(sol))
    sol = sol[i:] + sol[:i]  # rotation
    if sol[1] > sol[-1]:  # right > left
        sol = [sol[0]] + sol[:0:-1]
    return sol
```

```
A  = [C2, C4, C1, C3, C0, C5]
A' = [C5, C0, C3, C1, C4, C2]
(A' is A read backwards)

normalize(A) = normalize(A')
  = [C0, C3, C1, C4, C2, C5]
```

Fitness

- Sum of the differences between adjacent children
- Each pair counted once; the last child is next to the first

```
def fitness(solution):
    sol = list(solution)
    return sum(difference(c, cn) for c, cn in
               zip(sol, sol[1:] + sol[:1]))
```

Example with 5 children

```
A-B 3, B-C 5, C-D 2, D-E 4, E-A 6
fitness = 3 + 5 + 2 + 4 + 6 = 20
```

Two tweaks

Swap

Swap two children

```
before:  A B C D E F
after:   A E C D B F
```

up to 4 adjacent pairs change

Inversion

Reverse a segment

```
before:  A [B C D E] F
after:   A [E D C B] F
```

only 2 adjacent pairs change

Both return a normalized copy

Crossover: order crossover (OX)

```
parent 1:   0 3 [1 4 7] 2 6 5
parent 2:   0 1  2 3 4  5 6 7
child:      0 2 [1 4 7] 3 5 6
```

```
segment = list(p1[i:j])
taken = set(segment)
rest = [c for c in p2 if c not in taken]
return normalize(rest[:i] + segment + rest[i:])
```

- Keep a segment of parent 1 in place
- Fill the rest in the order of parent 2
- Always a legal permutation

Hill climbing

- One solution: tweak it, keep the new one if it is not worse
- >= so it can move on plateaus
- Stops at the budget of evaluations, or after patience evaluations without improvement
- Baselines: a random circle, and the greedy circle

```
candidate = tweak_function(current)
candidate_f = fitness(candidate)
if candidate_f >= current_f:
    current = candidate
    current_f = candidate_f
if evals - last_improvement >= patience:
    break
```

greedy: start from Child 0 and always put next the child that differs most from the last one

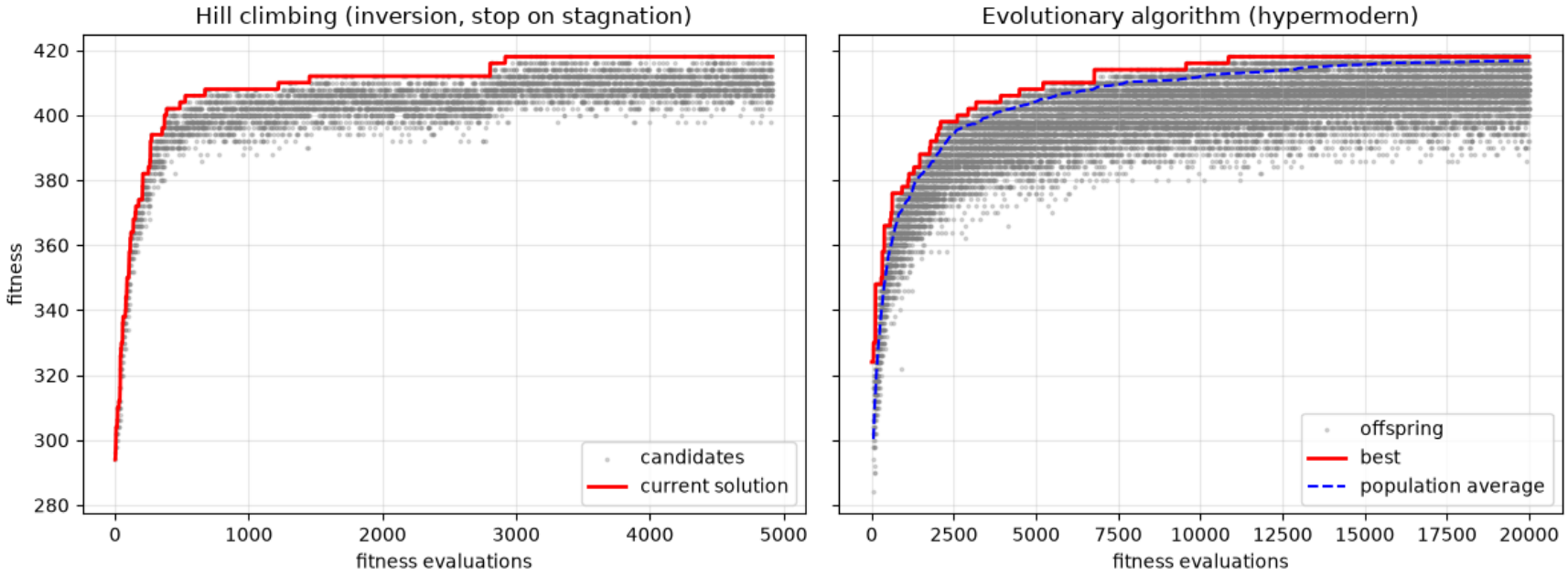
Evolutionary algorithm

- **Tournament:** best of 3 individuals, drawn with replacement
- **Hypermodern cycle:** choose the operator, then its parents, then apply it
- **(mu + lambda):** 50 parents + 30 new individuals compete, the best 50 survive; equal circles are kept once

```
OPERATORS = [
  (order_crossover, 2, 0.4),
  (tweak_inversion, 1, 0.4),
  (tweak_swap,      1, 0.2)]

op = choices(functions, weights)[0]
parents = [tournament(population,
    fitnesses) for _ in range(arity[op])]
offspring.append(op(*parents))
```

One run each, 30 children



Hill climbing stops after 4,918 evaluations (patience = 2000) with fitness 418. Evolutionary algorithm: 418 after 20,000.
Grey: every candidate, red: best, blue: population average.

Comparison

Mean final fitness, 10 runs	**n = 30**	**n = 100**
random circle	299.9	1002.2
greedy	410.0	1480.0
hill climbing, swap	406.4	1410.0
hill climbing, inversion	418.0	1478.0
evolutionary algorithm	416.6	1410.8

20,000 evaluations per run, 20 characteristics. random: mean of 200 circles. greedy: deterministic.

Conclusions

- **The tweak matters:** inversion beats swap (418.0 vs 406.4 with 30 children, 1478 vs 1410 with 100)
- **Greedy is a strong baseline:** one solution, no search, already 410 and 1480
- **The evolutionary algorithm does not beat hill climbing with inversion:** 416.6 vs 418.0 with 30 children, 1410.8 vs 1478.0 with 100
    - with the same 20,000 evaluations, the EA spreads them over a population of 50 solutions (about 400 per individual on average), while the climber spends all of them on one